Real-Time Face Mask Detection System Using MobileNetV2 and OpenCV

Rohail Ahmad — AI & Machine Learning Engineer

Computer Vision & Deep Learning Application — 2023

github.com/rohailahmad-code/Face-Mask-Detection

ABSTRACT

The COVID-19 pandemic underscored the critical importance of face mask compliance in public spaces as a primary non-pharmaceutical intervention. This report presents a real-time face mask detection system built using **MobileNetV2** — a lightweight depthwise separable convolutional neural network — combined with **OpenCV's** DNN-based Caffe face detector for precise face localization. The system classifies detected faces as *Mask* or *No Mask* in live webcam or CCTV video streams with high accuracy and low latency, making it suitable for edge deployment. The model was trained on a curated Kaggle dataset of 3,835 images and achieves over **98% validation accuracy** with real-time inference at over 25 FPS.

Keywords: Face Mask Detection, MobileNetV2, OpenCV, TensorFlow, Keras, Transfer Learning, Real-Time Computer Vision, COVID-19, Edge Deployment.

98.2%	25+ FPS	3,835	2-Class
Validation Accuracy	Real-time Inference	Training Images	Mask / No Mask

1. Introduction

The rapid global spread of COVID-19 brought renewed attention to the enforcement of mask-wearing policies in public settings. Manual monitoring of mask compliance is labor-intensive and impractical at scale in environments such as airports, hospitals, schools, and shopping centers. Automated computer vision systems offer an efficient, scalable, and contactless alternative.

This project implements a complete end-to-end pipeline: face detection using a pre-trained OpenCV Caffe model, followed by mask/no-mask classification using a fine-tuned MobileNetV2. The system draws color-coded bounding boxes (green for Mask, red for No Mask) with confidence scores on live video frames, enabling real-time visual alerts. The architecture was selected for its balance between accuracy and computational efficiency, enabling deployment on standard CPU hardware without a GPU.

2. Related Work

Prior work on face mask detection broadly falls into two categories: *object detection approaches* (YOLO, SSD, Faster R-CNN) and *classification-on-crop approaches* (MobileNet, ResNet, VGG). Loey et al. (2021) demonstrated that MobileNetV2 outperforms VGG16 in terms of inference speed while maintaining competitive accuracy for mask detection tasks. Sethi & Kathuria (2021) applied ResNet-50 and achieved 99.64% accuracy but at the cost of 3× higher inference time compared to MobileNetV2. OpenCV's DNN module has been widely used for face localization in production systems due to its cross-platform compatibility and C++/Python API.

This project follows the classification-on-crop paradigm, aligning with Loey et al.'s findings that MobileNetV2 provides the optimal accuracy-to-speed trade-off for real-time mask detection.

3. Dataset

The dataset used in this project is sourced from the **Kaggle Face Mask Detection Dataset**, which contains annotated images of people wearing masks, not wearing masks, and wearing masks incorrectly. For binary classification, images were organized into two classes: `with_mask` and `without_mask`.

1,916	1,919	3,835
Images ✓ **With Mask**	Images ✗ **Without Mask**	Total Images **Balanced Dataset**

Data Preprocessing & Augmentation

- **Resizing:** All images resized to 224×224 pixels (MobileNetV2 input requirement)
- **Normalization:** Pixel values scaled to [0, 1] using ImageNet mean subtraction
- **Augmentation (training only):**
  - Random rotation: ±20°
  - Horizontal flip (50% probability)
  - Width/height shift: up to 20%
  - Shear transformation: up to 20%
  - Zoom: up to 15%
- **Split:** 80% training / 20% validation (stratified)

4. System Architecture



The pipeline operates on each video frame independently. OpenCV's pre-trained Caffe face detector (`res10_300x300_ssd_iter_140000.caffemodel`) identifies face regions with associated confidence scores. Detected faces with confidence > 50% are cropped, resized, and passed through the MobileNetV2 classifier. The output probability is used to assign a Mask or No Mask label, which is overlaid on the original frame with a color-coded bounding box.

5. Model: MobileNetV2 with Transfer Learning

MobileNetV2 Overview

MobileNetV2 (Sandler et al., 2018) is an efficient CNN architecture designed for mobile and edge devices. Its key innovations are **inverted residuals** (expanding low-dimensional feature maps before applying depthwise separable convolutions) and **linear bottlenecks** (preserving information by using linear rather than ReLU activations in bottleneck layers). This reduces parameters dramatically while maintaining representational power — the model achieves 72.0% top-1 accuracy on ImageNet with only 3.4M parameters.

Fine-Tuning Strategy

Transfer learning from ImageNet-pretrained MobileNetV2 weights was applied following a two-phase training strategy:

1. **Phase 1 — Head Training:** The MobileNetV2 base was frozen; only the newly added classification head (AveragePooling2D → Flatten → Dense(128, ReLU) → Dropout(0.5) → Dense(2, Softmax)) was trained

for 20 epochs.

2. **Phase 2 — Fine-Tuning:** The top 30 layers of MobileNetV2 were unfrozen and trained at a low learning rate (1e-4) for an additional 10 epochs to adapt higher-level features to the mask detection domain.

Table 1: Training Hyperparameters

Parameter	Phase 1 (Head)	Phase 2 (Fine-tune)
Optimizer	Adam	Adam
Learning Rate	1e-3	1e-4
Batch Size	32	32
Epochs	20	10
Loss Function	Binary Cross-Entropy	Binary Cross-Entropy
Frozen Layers	All base layers	Top 30 unfrozen
Dropout	0.5	0.5

6. Experimental Results

Classification Performance

Table 2: Model Performance on Validation Set

Metric	With Mask	Without Mask	Overall
Precision	**98.5%**	**97.9%**	**98.2%**
Recall	97.8%	98.6%	98.2%
F1-Score	98.1%	98.2%	98.2%
Support	383	384	767

Inference Performance

Table 3: Runtime Performance Benchmarks

Hardware	FPS	Latency per Frame	Notes
Intel Core i5 CPU (no GPU)	**28 FPS**	~36ms	Real-time capable
NVIDIA GTX 1650 GPU	**60+ FPS**	<17ms	Smooth streaming
Raspberry Pi 4 (edge)	8 FPS	~125ms	Lightweight deployment

[Training Accuracy & Loss Curves]
Phase 1 converged by epoch 15 · Phase 2 fine-tuning improved validation accuracy by ~1.4%

Figure 1: Training and Validation Accuracy/Loss over 30 epochs (Phase 1 + Phase 2 combined).

Confusion Matrix Insights

The confusion matrix analysis revealed that the model's primary error mode is false negatives — classifying a masked face as unmasked — which occurred in cases where masks were incorrectly worn (e.g., below the nose) or where the face was partially occluded. These edge cases account for <2% of the validation set and represent a known challenge in binary mask classification.

7. System Features

- **Real-time mask detection** from webcam or CCTV feeds at 25+ FPS
- **Binary classification:** Mask vs. No Mask with confidence scores displayed
- **Color-coded bounding boxes:** Green (Mask) / Red (No Mask) for immediate visual feedback
- **Multi-face detection:** Handles multiple faces per frame simultaneously
- **Confidence thresholding:** Configurable detection confidence (>50%) to reduce false positives
- **Minimal setup:** Single Python script with `requirements.txt`; runs on CPU without a GPU
- **High accuracy:** 98.2% validation accuracy with robust augmentation and fine-tuning

8. Technologies & Stack

Table 4: Technology Stack

Category	Tool / Library	Version	Purpose
Language	Python	3.8+	Core implementation language
Deep Learning	TensorFlow / Keras	2.x	Model training & inference
Architecture	MobileNetV2	ImageNet weights	Transfer learning backbone
Computer Vision	OpenCV	4.x	Face detection, video I/O, frame rendering
Face Detector	Caffe DNN (SSD)	res10_300x300	Pre-trained face localization
Numerical	NumPy	1.21+	Array ops, preprocessing
Data Handling	imutils	0.5.4	Video stream utilities
Visualization	Matplotlib	3.x	Training curve plots

9. Deployment Pipeline

1. **Model Training:** Run `train_mask_detector.py` to fine-tune MobileNetV2 on the dataset. The trained model weights are serialized to `mask_detector.model`.
2. **Live Detection:** Execute `detect_mask_video.py` to start the real-time pipeline. OpenCV opens the default webcam, processes each frame, and overlays predictions.
3. **Image Detection:** Run `detect_mask_image.py --image <path>` for single-image inference — useful for batch processing of surveillance snapshots.
4. **Edge Deployment:** The serialized model can be exported as a TFLite file (`mask_detector.tflite`) for deployment on Raspberry Pi or ARM-based edge devices.

> **Deployment Note:** The system requires only Python 3.8+, OpenCV, and TensorFlow to run. No GPU is needed for inference — the model runs comfortably in real time on standard CPU hardware, making it accessible for budget-constrained deployment scenarios.

10. Discussion

Why MobileNetV2?

Alternative architectures considered included VGG16, ResNet-50, and EfficientNetB0. MobileNetV2 was selected for three key reasons: (1) its depthwise separable convolutions reduce computational cost by ~8× compared to standard convolutions, enabling real-time CPU inference; (2) ImageNet pretraining provides excellent low-level feature priors (edges, textures) that transfer well to face/mask patterns; and (3) its compact model size (~14 MB) makes it suitable for edge deployment.

Limitations

- Binary classification only — does not detect *incorrectly worn* masks as a separate class
- Performance degrades under low-light conditions or extreme pose angles (>45°)
- Face detector struggles with very small faces (<30×30 pixels) in wide-angle footage
- Dataset was balanced but collected under controlled conditions — may not cover all ethnicities and mask types equally

11. Future Enhancements

→ 3-class detection: Mask / No Mask / Worn Incorrectly	→ YOLO-based detector for faster multi-face localization
→ Night-vision / IR camera compatibility for 24/7 operation	→ Alert system integration (buzzer, email, SMS notifications)
→ TFLite export for Raspberry Pi and Jetson Nano deployment	→ Crowd density estimation with mask compliance rate reporting

12. Conclusion

> This project demonstrates that a fine-tuned MobileNetV2 classifier, combined with OpenCV's DNN face detector, delivers a practical, high-accuracy (98.2%) real-time face mask detection system that runs efficiently on CPU hardware. The two-phase transfer learning approach — head training followed by selective fine-tuning — proved effective in adapting ImageNet features to the mask detection domain. The system is deployable with minimal hardware requirements and serves as a strong baseline for production-grade mask compliance monitoring in public settings.

13. References

[1] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *CVPR 2018*, pp. 4510–4520.

[2] M. Loey, G. Manogaran, M. H. N. Taha, and N. E. M. Khalifa, "A hybrid deep transfer learning model with machine learning methods for face mask detection in the era of the COVID-19 pandemic," *Measurement*, vol. 167, 2021.

[3] S. Sethi and R. Kathuria, "Face Mask Detection using Deep Learning: An Approach to Reduce Risk of Coronavirus Spread," *Journal of Biomedical Informatics*, vol. 120, 2021.

[4] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[5] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint Face Detection and Alignment Using Multitask Cascaded Convolutional Networks," *IEEE Signal Processing Letters*, vol. 23, no. 10, 2016.

[6] World Health Organization, "Coronavirus disease (COVID-19): Masks," WHO Technical Report, 2021.

[7] A. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," *arXiv:1704.04861*, 2017.

© 2023 Rohail Ahmad · Computer Vision & Deep Learning Application · All rights reserved · GitHub Repository